

NOVAC Internship – Task 5 Daily Documentation

Task: Vector Database Integration, Semantic Search, PaddleOCR Integration, and Advanced Semantic Chunking

Overview

Today's work focused on transforming the existing AI document ingestion pipeline into a Retrieval-Augmented Generation (RAG)-ready architecture by integrating vector database storage, semantic search functionality, advanced OCR processing, and improved semantic chunking strategies.

The previously developed FastAPI-based ingestion system was enhanced to support semantic vector retrieval workflows using embeddings generated through Sentence Transformers. The project evolved from simple document storage into a semantic document intelligence pipeline capable of retrieval-oriented processing.

The task involved understanding vector databases, semantic search workflows, embedding-based retrieval systems, and modern OCR frameworks. The OCR pipeline was upgraded from Tesseract OCR to PaddleOCR for improved multilingual recognition accuracy and modern deep-learning-based text extraction.

By the end of the session, the system was successfully able to:

- upload TXT, PDF, DOCX, and image files
- extract text using OCR
- generate semantic embeddings
- store embeddings and chunks inside MongoDB and vector storage
- perform semantic search
- dynamically display editable chunks on the frontend
- improve semantic chunking behavior for OCR-generated content

Resources and References Used

- PaddleOCR Documentation
- Sentence Transformers Documentation
- ChromaDB Documentation
- FastAPI Official Documentation
- MongoDB Documentation
- Scikit-learn Cosine Similarity Documentation
- NLP Sentence Tokenization Concepts
- Retrieval-Augmented Generation (RAG) Architecture Concepts

Concepts Learned

Retrieval-Augmented Generation (RAG)

RAG is an AI architecture that combines:

- vector retrieval
- semantic search
- Large Language Models (LLMs)

to retrieve relevant contextual information before generating responses.

Learned how RAG pipelines work:

Document Upload



Chunking



Embeddings

↓
Vector Database
↓
Semantic Retrieval
↓
LLM Context Injection

Vector Databases

Learned how vector databases store embeddings instead of raw text and enable semantic retrieval through vector similarity.

Understood how embeddings are used for:

- semantic search
- context retrieval
- document intelligence systems
- chatbot memory systems

Semantic Search

Semantic search retrieves documents based on meaning rather than keyword matching.

Implemented semantic retrieval using:

- Sentence Transformers embeddings
- cosine similarity
- vector comparison workflows

PaddleOCR

Studied PaddleOCR as a modern deep-learning-based OCR framework with:

- multilingual support
- improved text recognition
- layout-aware extraction
- advanced OCR pipelines

Learned how PaddleOCR differs from traditional OCR systems like Tesseract.

Advanced Semantic Chunking

Improved chunking strategies for OCR-generated text by implementing:

- semantic similarity-based sentence merging
- larger contextual chunk grouping
- word-count-aware chunk balancing
- OCR text cleanup and normalization

Libraries, Modules, and Components Learned

PaddleOCR: Used for advanced OCR-based text extraction from uploaded image files.

Sentence Transformers: Used for generating semantic embeddings.

ChromaDB: Used as a vector database for semantic retrieval workflows.

Scikit-learn: Used for cosine similarity calculations.

NLTK: Used for sentence tokenization, word tokenization

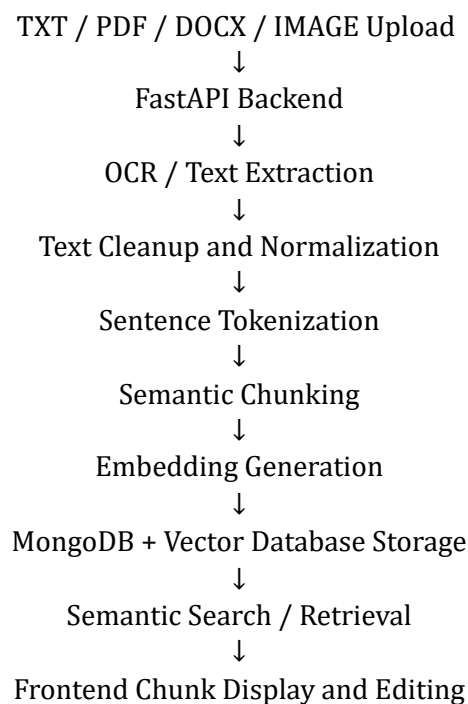
MongoDB: Used for storing chunks, metadata, embeddings.

FastAPI: Used for backend API development.

Practical Implementation

- Installed PaddleOCR and dependencies
- Integrated PaddleOCR into FastAPI backend
- Added semantic search functionality
- Implemented vector database workflows
- Stored embeddings inside vector storage
- Improved OCR extraction logic
- Fixed OCR response parsing issues
- Added OCR cleanup and normalization
- Improved semantic chunk balancing
- Implemented word-aware semantic chunking
- Added frontend chunk editing support
- Improved frontend UI styling using basic CSS
- Center-aligned frontend layout
- Added upload container UI
- Added semantic retrieval preparation architecture

Architecture Understood



Challenges Faced

Several implementation and dependency issues were encountered during integration.

PaddleOCR Compatibility Issues

- deprecated API parameters
- OCR response structure differences
- newer PaddleOCR parsing changes

Semantic Chunking Issues

- over-fragmented chunks
- OCR-generated noisy tokens
- uneven chunk balancing

OCR Noise

OCR extraction produced:

- incorrect punctuation
- character artifacts
- malformed words

Additional cleanup logic was required to normalize extracted text.

Dependency Warnings

Version mismatches occurred between:

- requests
- urllib3
- packaging
- OCR dependencies

Conclusion

Today's work significantly upgraded the project from a simple AI ingestion pipeline into a foundational RAG-ready semantic document intelligence system.

The system can now:

- process multimodal documents
- perform OCR extraction
- generate embeddings
- store vector representations
- support semantic retrieval workflows
- display editable chunks dynamically on the frontend

The concepts learned today form the foundation of:

- semantic document retrieval systems
- vector search engines
- enterprise RAG pipelines
- AI-powered document assistants
- multimodal knowledge systems

Prepared By

Divesh Kumar S